Développement d'applications Framework JavaFX

Chapitre 2

TP Bibliothèque

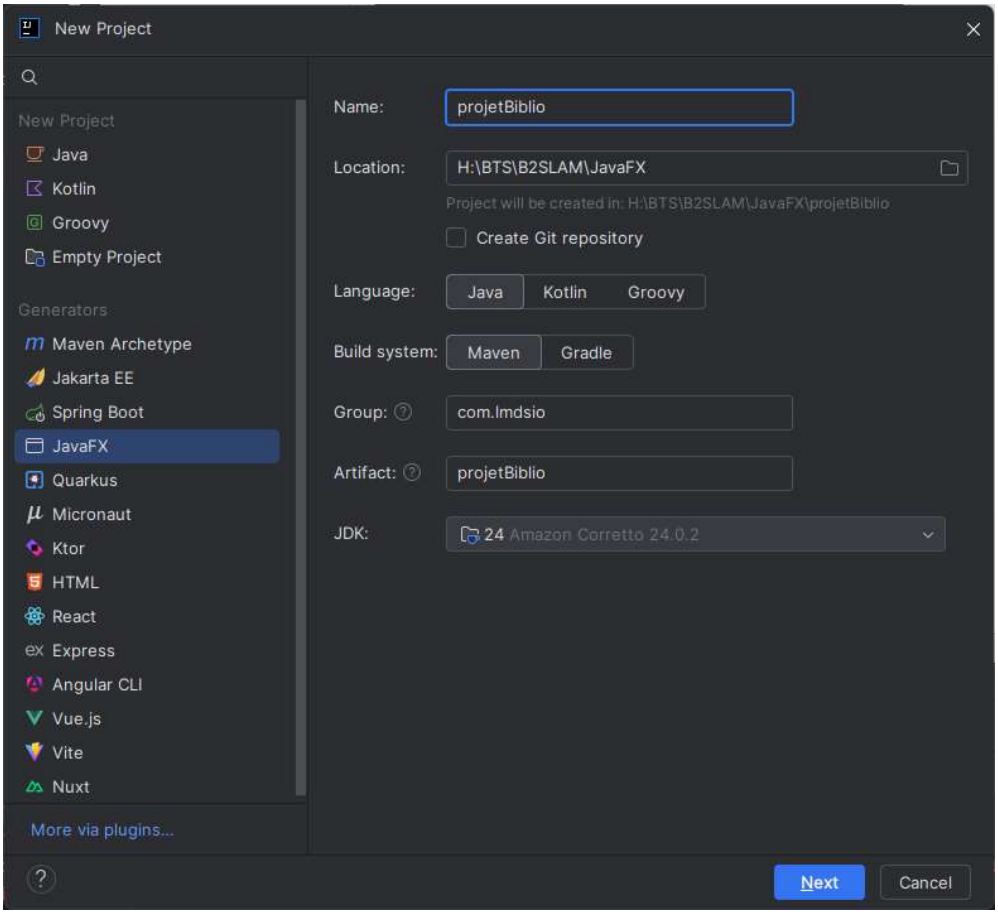
Introduction

Nous allons créer un projet JavaFX plus complet, avec une architecture pour séparer les données (modèles métier), leur stockage (via des services), leur traitement (dans les contrôleurs) et leur affichage (dans les vues).

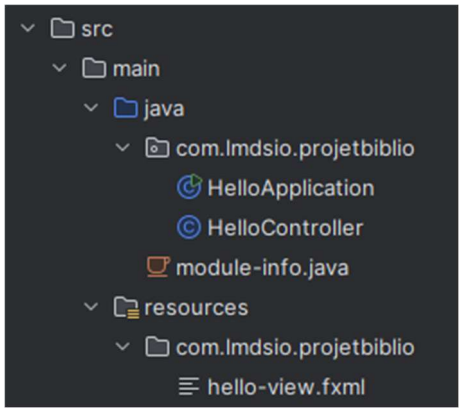
Le but est d'implémenter une application de gestion de bibliothèque, avec un catalogue des livres disponibles, la liste des usagers et le suivi des emprunts. Vous allez notamment reprendre les notions de Programmation Orientée Objet vues précédemment. De plus, l'application se connectera à un serveur de base de données.

Configuration de l'application

Depuis IntelliJ, créez dans votre dossier personnel un nouveau projet JavaFX nommé projetBiblio



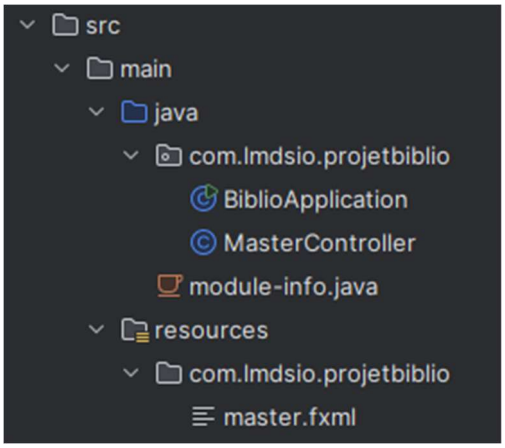
N'ajoutez pas de librairies supplémentaires et générez l'application avec les fichiers par défaut :



Vous allez tout d'abord renommer les fichiers et organiser l'architecture du projet.

Avec un clic droit sur les fichiers, puis Rename :

- HelloApplication devient **BiblioApplication** puis Refactor
- HelloController devient **MasterController** puis Refactor, et Refactor encore pour mettre à jour le lien vers le contrôleur dans le fichier fxml
- Hello-view.fxml devient **master.fxml** puis Refactor, et Refactor encore pour mettre à jour le chargement de la vue dans le contrôleur

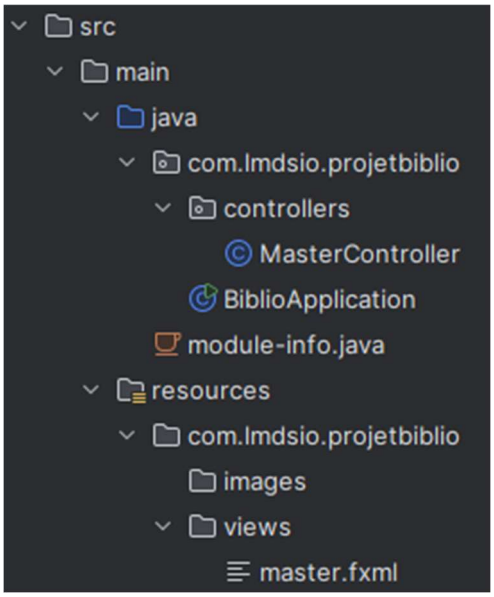


Avec un clic droit sur le dossier com.lmdsio.projetbiblio dans java, puis New Package :

- Créez le package **com.lmdsio.projetbiblio.controllers**
- Faites glisser le fichier MasterController dans ce package controllers, puis Refactor et Refactor encore

Avec un clic droit sur le dossier com.lmdsio.projetbiblio dans resources, puis New Directory :

- Créez le dossier **views**
- Créez le dossier **images**
- Faites glisser le fichier master.fxml dans le dossier views puis Refactor



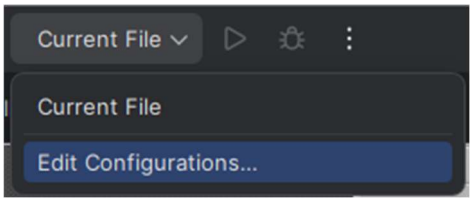
Dans la méthode start de BiblioApplication, modifiez le chargement de la vue master depuis le dossier views :

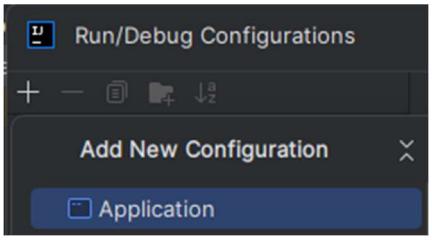
```
FXMLLoader fxmlLoader = new FXMLLoader(BiblioApplication.class.getResource("views/master.fxml"));
```

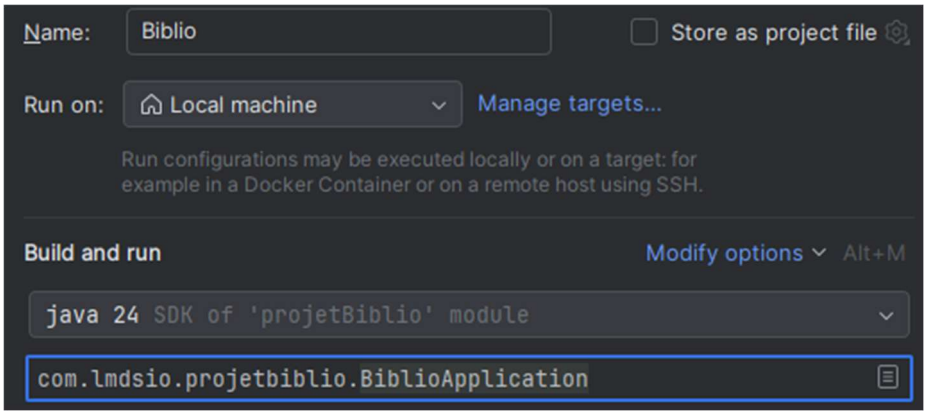
Et modifiez le titre de la fenêtre principale :

```
stage.setTitle("Biblio 1.0");
```

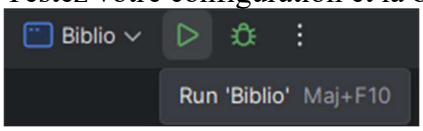
Ajoutez une configuration d'exécution pour l'application, en spécifiant la classe principale BiblioApplication:







Testez votre configuration et la bonne exécution de l'application :



Enfin, personnalisez l'icône de l'application (visible dans la fenêtre et dans la barre des tâches) :

- Téléchargez une icône au format png depuis un site avec des images libres de droits (par exemple https://www.freepik.com/icons ou https://iconmonstr.com)
- Placez le fichier png dans le dossier images des ressources
- Ajoutez le chargement du fichier png dans la méthode start de BiblioApplication

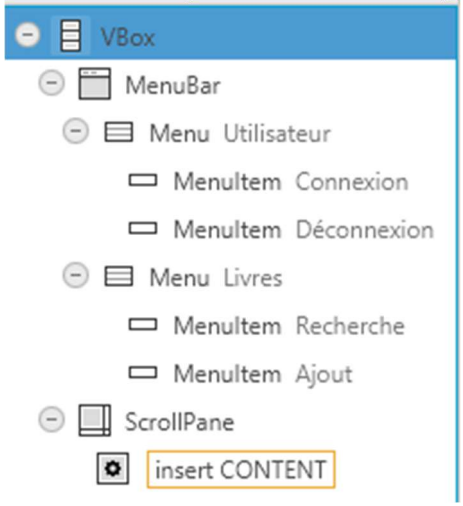
```
stage.getIcons().add(new Image(BiblioApplication.class.getResourceAsStream("images/book.png")));
stage.setScene(scene);
```

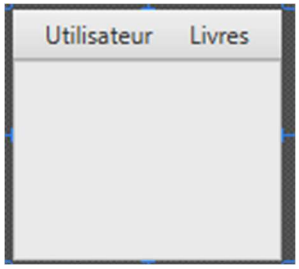
Page d'accueil

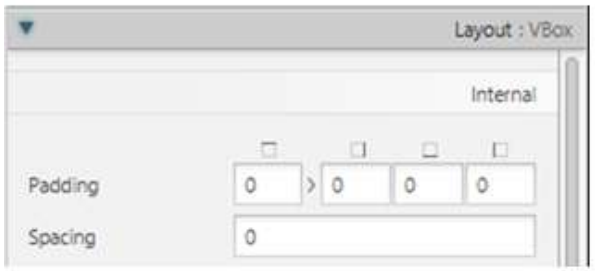
Layout master

Nous allons mettre en place une vue principale (layout), qui contiendra le menu de l'application, ainsi qu'une zone pour insérer différents contenus. Les menus pourront par exemple modifier le contenu affiché.
Il s'agit de la vue master.fxml, gérée par son contrôleur MasterController.

Modifiez complètement cette vue pour arriver au design ci-dessous :





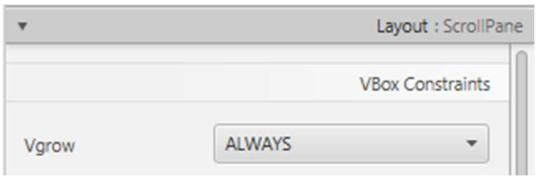


Le conteneur principal est toujours un **VBox**, mais sans aucune marge ou espacement, avec alignement en haut.

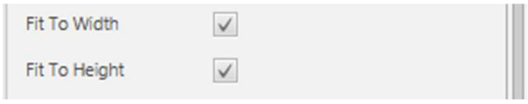
Effacez tout à l'intérieur, puis placez un contrôle **MenuBar** avec deux **Menu** (modifiez ceux par défaut) :

- Un premier menu « Utilisateur », avec deux **MenuItem** « Connexion » et Déconnexion »
- Donnez-leur les identifiants **fx:id** `menuConnexion` et `menuDeconnexion`
- Un deuxième menu « Livres », avec deux **MenuItem** « Recherche » et « Ajout »

Placez en dessous de la barre de menus un conteneur **ScrollPane** avec l'identifant `contentPane`, et les propriétés suivantes :

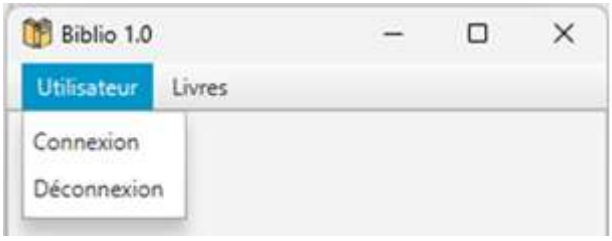


Pour forcer le ScrollPane à remplir entièrement son parent VBox



Pour forcer le contenu à remplir entièrement son parent ScrollPane

Ce qui donne comme rendu, le menu en haut de la fenêtre, avec en dessous une zone pour le contenu, vide pour l'instant :



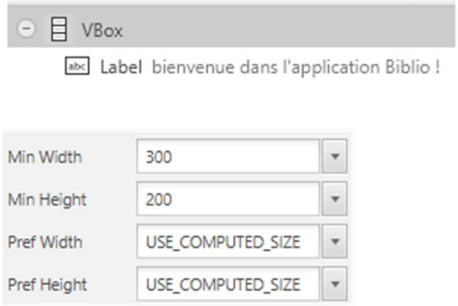
Le contenu du ScrollPane sera affiché grâce à une fonction showContent dans MasterController.
Cette fonction va charger un fichier fxml, pour le placer ensuite à l'intérieur du ScrollPane contentPane.

Recodez le MasterController :

```java
MasterController.java
package com.lmdsio.projetbiblio.controllers;

import com.lmdsio.projetbiblio.BiblioApplication;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Node;
import javafx.scene.control.ScrollPane;

public class MasterController {
    @FXML
    private ScrollPane contentPane;

    protected void showContent(String fxmlName) {
        Node contentNode;
        try {
            FXMLLoader fxmlLoader =
                    new FXMLLoader(BiblioApplication.class.getResource("views/" + fxmlName + ".fxml"));
            contentNode = fxmlLoader.load();
        } catch (Exception e) {
            contentNode = null;
        }
        contentPane.setContent(contentNode);
    }
}
```

Vue welcome

Avec un clic droit sur le dossier views, puis New FXML File, créez une nouvelle vue welcome.fxml.
Par défaut la vue est créée avec un conteneur AnchorPane.
Supprimez-le et placez un VBox à la racine de la vue, avec à l'intérieur un Label qui affiche « Bienvenue dans l'application Biblio ! ».
Choisissez l'alignement centré pour le VBox, afin d'avoir le Label au milieu.
Réduisez la taille du VBox à 300 par 200 pixels minimum.

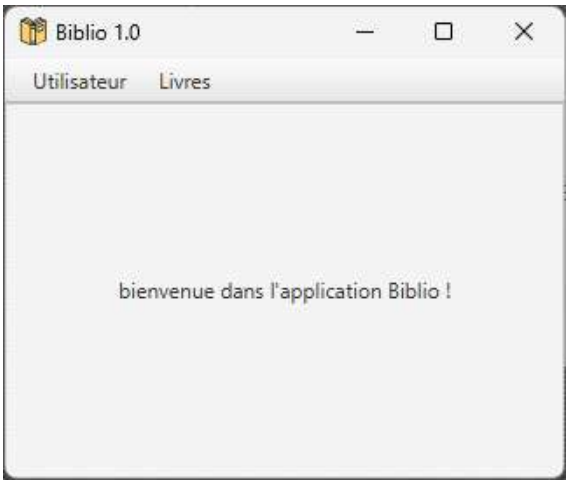


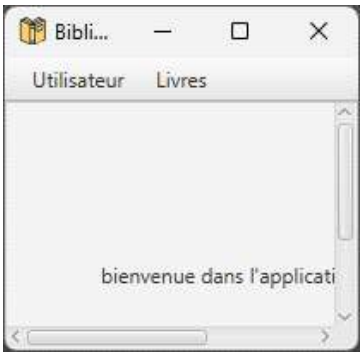
Ajoutez maintenant la fonction suivante dans Mastercontroller :

```java
@FXML
protected void initialize() {
    showContent( fxmlName: "welcome");
}
```

La méthode **initialize** du contrôleur est appelée automatiquement par le framework lorsque la scène est créée.

Testez l'affichage, notamment en modifiant la taille de la fenêtre principale :

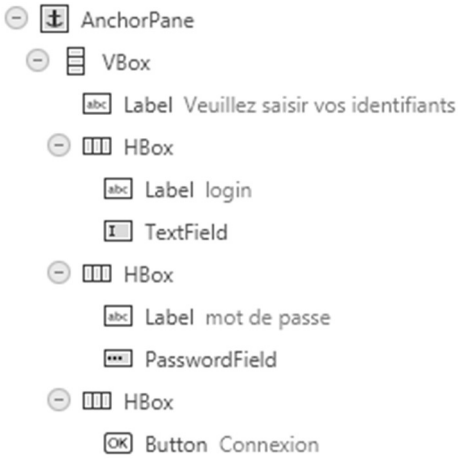


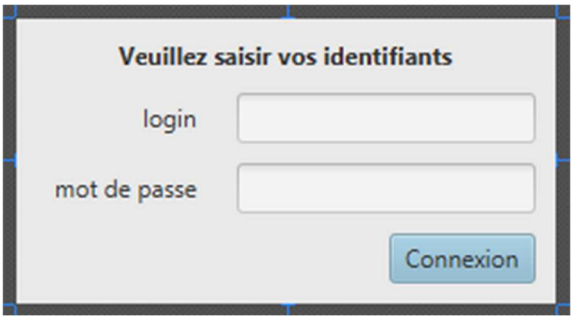


Page de connexion

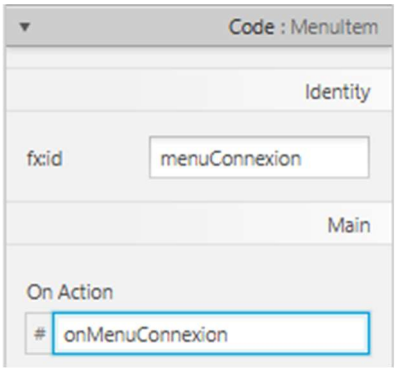
Vue login

Créez maintenant une deuxième vue login.fxml.
A l'intérieur du conteneur AnchorPane, construisez un formulaire d'authentification sur la base du TP précédent (avec les identifiants `loginText`, `loginField` et `passwordField` pour les contrôles) :





Nous voulons afficher ce formulaire quand on clique sur le menu Connexion.
Pour cela, ajoutez la fonction onMenuConnexion dans MasterController, déclenchée par le menu :

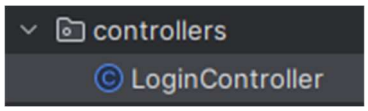



```
@FXML
protected void onMenuConnexion() {
    showContent("login");
}
```

Contrôleur LoginController

Nous avons besoin d'un contrôleur pour interagir avec le formulaire.

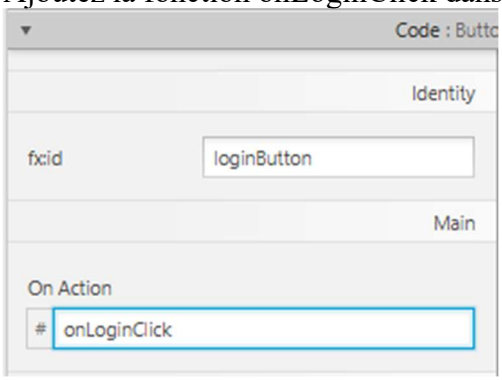
Avec un clic droit sur le dossier controllers, puis New Java Class, créez le contrôleur LoginController.



Dans le fichier login.fxml, faîtes le lien vers ce contrôleur :

```
<AnchorPane xmlns="http://javafx.com/javafx/17.0.12" xmlns:fx="http://javafx.com/fxml/1"
            fx:controller="com.lmdsio.projetbiblio.controllers.LoginController">
```

Ajoutez la fonction onLoginClick dans les propriétés du bouton :

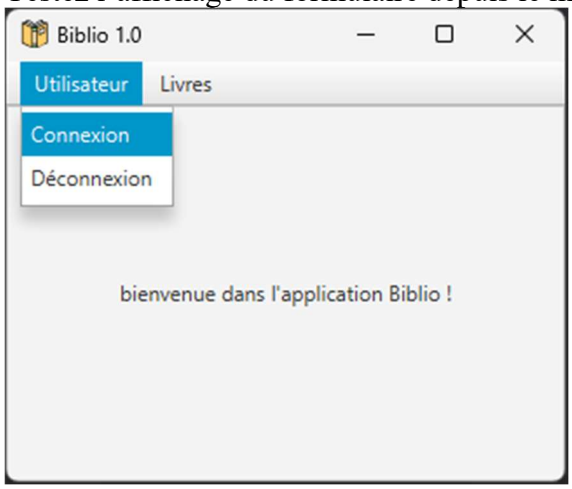


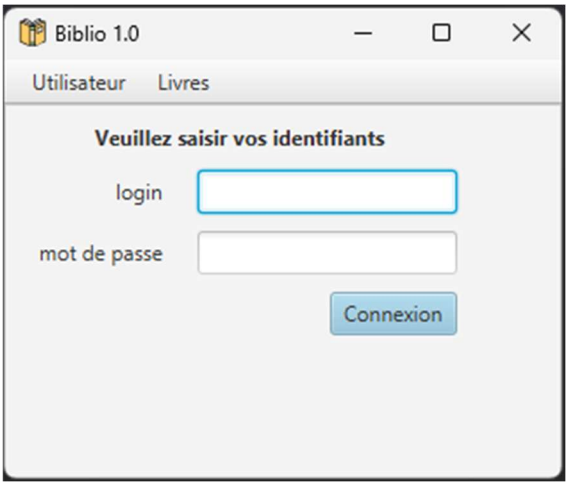
Puis codez le contrôleur, avec une première version basique de l'authentification :

LoginController.java

```
1   package com.lmdsio.projetbiblio.controllers;
2
3   import javafx.fxml.FXML;
4   import javafx.scene.control.*;
5
6   public class LoginController {
7       @FXML
8       private Label loginText;
9       @FXML
10      private TextField loginField;
11      @FXML
12      private PasswordField passwordField;
13
14      @FXML
15      private void onLoginClick() {
16          String login = loginField.getText();
17          String password = passwordField.getText();
18          if (password.equals("mdp")) {
19              loginText.setText(" Hello " + login + " ! ");
20              loginText.setStyle("-fx-text-fill: green");
21          } else {
22              loginText.setText("Identifiants incorrects !");
23              loginText.setStyle("-fx-text-fill: red");
24          }
25      }
26  }
```

Testez l'affichage du formulaire depuis le menu Connexion.





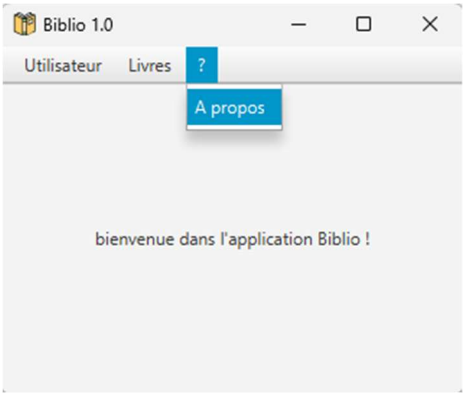
Et le fonctionnement de l'authentification avec le mot de passe « mdp »

Exercice d'application

D'après les images ci-dessous, ajoutez un menu « A propos » :

- qui affichera une nouvelle vue **about.fxml** (liée à son contrôleur **AboutController**)
- avec une image de votre choix dans un contrôle **ImageView**

- avec un contrôle **ToggleButton** pour montrer/masquer les crédits (à votre nom)



⇨ **Déposez un fichier zip de votre projet**